

# HPSC 2026 Course Assignment 1

## OpenMP parallelisation of particle simulator

Gaurav Bhutani

March 18, 2026

### 1 Instructions

- You are allowed to use LLMs but report the use and ensure that you have checked everything for correctness and you understand everything.
- All figures must be publication quality with fontsize matching the main text.
- All codes, scripts and Latex support files must be uploaded on Github with a link in the report pdf.
- You may be asked to explain your work (in the form of a viva) if we feel that LLMs have been used without much thought.
- The assignment is for 5 marks, however you may be able to earn up to 7 additional marks through bonus work.
- In case you are aiming 1 or 2 extra bonus marks, specify the results clearly in the report. Present the algorithms, analyses and plots corresponding to the additional work in detail.
- In case you are going for the additional 5 marks corresponding to the research paper, submit the detailed paper as instructed in the relevant section below.

### 2 Objective

The goal of this assignment is to develop, verify, profile and parallelise a three-dimensional Discrete Element Method (DEM) solver for spherical particles. The solver will compute only the **translational motion** of particles and ignore rotational dynamics.

Students will implement the algorithm in **Fortran or C++** and then parallelise the computationally intensive portions of the code using **OpenMP**. The exercise follows a typical scientific computing workflow:

1. Mathematical formulation of the problem
2. Serial implementation of the algorithm
3. Verification and testing

4. Code profiling
5. Parallelisation using OpenMP
6. Performance analysis and scaling study

The assignment culminates in a short technical **white paper** describing your implementation and results.

### 3 Physical Model

We consider  $N$  spherical particles moving inside a rectangular box domain.

Each particle  $i$  has

- position  $\mathbf{x}_i = (x_i, y_i, z_i)$
- velocity  $\mathbf{v}_i = (v_{x,i}, v_{y,i}, v_{z,i})$
- mass  $m_i$
- radius  $R_i$

The motion of each particle is governed by Newton's second law.

$$m_i \frac{d\mathbf{v}_i}{dt} = \mathbf{F}_i \quad (1)$$

$$\frac{d\mathbf{x}_i}{dt} = \mathbf{v}_i \quad (2)$$

The total force on particle  $i$  is

$$\mathbf{F}_i = m_i \mathbf{g} + \sum_{j \neq i} \mathbf{F}_{ij}^c + \mathbf{F}_i^w \quad (3)$$

where

- $\mathbf{g}$  is gravitational acceleration
- $\mathbf{F}_{ij}^c$  is the particle–particle contact force
- $\mathbf{F}_i^w$  is the particle–wall contact force

### 4 Particle Contact Model

The contact force is computed using a simple linear spring-dashpot model.

Let

$$\mathbf{r}_{ij} = \mathbf{x}_j - \mathbf{x}_i \quad (4)$$

$$d_{ij} = ||\mathbf{r}_{ij}|| \quad (5)$$

$$\mathbf{n}_{ij} = \frac{\mathbf{r}_{ij}}{d_{ij}} \quad (6)$$

The overlap between particles is

$$\delta_{ij} = R_i + R_j - d_{ij} \quad (7)$$

A contact exists if

$$\delta_{ij} > 0 \quad (8)$$

The normal relative velocity is

$$v_{n,ij} = (\mathbf{v}_j - \mathbf{v}_i) \cdot \mathbf{n}_{ij} \quad (9)$$

The contact force is then defined as

$$\mathbf{F}_{ij}^c = (k_n \delta_{ij} - \gamma_n v_{n,ij}) \mathbf{n}_{ij} \quad (10)$$

where

- $k_n$  is the normal stiffness
- $\gamma_n$  is a damping coefficient

To prevent nonphysical attractive forces, you may enforce

$$F_n = \max(0, k_n \delta_{ij} - \gamma_n v_{n,ij}) \quad (11)$$

and then compute

$$\mathbf{F}_{ij}^c = F_n \mathbf{n}_{ij} \quad (12)$$

## 5 Wall Contact

The domain is a cuboidal box

$$0 \leq x \leq L_x, \quad 0 \leq y \leq L_y, \quad 0 \leq z \leq L_z$$

If a particle overlaps a wall, apply a force using the same spring-dashpot model.

Example: lower  $z$  wall

$$\delta = R_i - z_i \quad (13)$$

if  $\delta > 0$ .

## 6 Time Integration

Use a semi-implicit Euler time integration scheme.

$$\mathbf{v}_i^{n+1} = \mathbf{v}_i^n + \frac{\mathbf{F}_i^n}{m_i} \Delta t \quad (14)$$

$$\mathbf{x}_i^{n+1} = \mathbf{x}_i^n + \mathbf{v}_i^{n+1} \Delta t \quad (15)$$

## 7 Timestep Algorithm

The simulation advances in time using a sequence of discrete timesteps. At each timestep, forces are first evaluated using the particle positions and velocities at the current time level, and then the particle velocities and positions are updated.

Let the timestep size be  $\Delta t$ , and let the simulation advance from time  $t^n$  to  $t^{n+1} = t^n + \Delta t$ .

### Algorithm for one timestep

Given particle positions  $\mathbf{x}_i^n$  and velocities  $\mathbf{v}_i^n$  for all particles:

1. Set all particle forces to zero:

$$\mathbf{F}_i^n = \mathbf{0}$$

2. Add body force due to gravity:

$$\mathbf{F}_i^n \leftarrow \mathbf{F}_i^n + m_i \mathbf{g}$$

3. For each particle pair  $(i, j)$  with  $i < j$ :

- (a) Compute relative position

$$\mathbf{r}_{ij} = \mathbf{x}_j^n - \mathbf{x}_i^n$$

- (b) Compute centre distance

$$d_{ij} = \|\mathbf{r}_{ij}\|$$

- (c) Compute overlap

$$\delta_{ij} = R_i + R_j - d_{ij}$$

- (d) If  $\delta_{ij} > 0$ , evaluate the contact force and add equal and opposite contributions to particles  $i$  and  $j$

4. For each particle, check contact with each wall of the box. If overlap with a wall exists, compute the corresponding wall force and add it to the total force on that particle.

5. Update particle velocities using the semi-implicit Euler step:

$$\mathbf{v}_i^{n+1} = \mathbf{v}_i^n + \frac{\mathbf{F}_i^n}{m_i} \Delta t$$

6. Update particle positions:

$$\mathbf{x}_i^{n+1} = \mathbf{x}_i^n + \mathbf{v}_i^{n+1} \Delta t$$

7. Compute diagnostics such as kinetic energy, particle positions, maximum speed, or number of contacts.

8. Write output at selected timesteps.

## Overall simulation loop

The full simulation may therefore be written conceptually as

1. Initialise particle positions, velocities, radii and masses
2. Set material and numerical parameters ( $k_n$ ,  $\gamma_n$ ,  $\Delta t$ , total simulation time)
3. Repeat the timestep algorithm until the final simulation time is reached
4. Post-process and analyse results

## 8 Suggested Program Structure

You are encouraged to organise your code into modular functions.

Examples include

- `initialize_particles`
- `zero_forces`
- `add_gravity`
- `compute_particle_contacts`
- `compute_wall_contacts`
- `integrate_particles`
- `compute_kinetic_energy`
- `write_output`

You may design additional helper functions as needed. Also, you may prototype in Python and then move to a compiled code. In that case, share the prototype over Github too.

## 9 Verification Tests

Before attempting parallelisation, verify that the serial implementation is correct.

### 9.1 Test 1: Free Fall

Single particle with zero initial velocity.

Analytical solution:

$$z(t) = z_0 - \frac{1}{2}gt^2 \tag{16}$$

$$v_z(t) = -gt \tag{17}$$

Compare numerical and analytical results.

## 9.2 Test 2: Constant Velocity

Set gravity to zero and ensure the particle travels with constant velocity.

## 9.3 Test 3: Particle Bounce

Drop a particle onto the bottom wall and observe bouncing behaviour.

Verify that

- the particle does not penetrate excessively
- rebound height decreases with damping

# 10 Simulation Experiments

Run simulations with multiple particles in the box domain.

Investigate systems with increasing particle counts, for example

- $N = 200$
- $N = 1000$
- $N = 5000$

# 11 Required Plots

Your report must include plots such as

- particle trajectory in the free fall test
- numerical vs analytical solution
- error vs timestep
- bouncing particle height vs time
- kinetic energy vs time
- particle configuration snapshots

# 12 Code Profiling

Profile the serial code to determine where the computational cost is spent.

Report

- total runtime
- runtime distribution across major functions
- identification of computational bottlenecks

Use timers or profiling tools. Also, try switching on compiler optimizations to see improvement (before moving over to parallelization).

## 13 Parallelisation

Parallelise the code using OpenMP.

Your goal is to improve performance of the most computationally intensive sections of the program.

Verify that

- parallel results match serial results
- numerical correctness is preserved

## 14 Performance Analysis

Evaluate performance for different thread counts.

Compute speedup

$$S(p) = \frac{T_1}{T_p} \quad (18)$$

and efficiency

$$E(p) = \frac{S(p)}{p} \quad (19)$$

where  $p$  is the number of threads.

Include

- runtime tables
- speedup plots
- efficiency plots

## 15 White Paper Submission

You must submit a short **white paper** (4–6 pages) describing your work.

The paper must be prepared using the **IEEE conference LaTeX template**.

Template available at

<https://www.ieee.org/conferences/publishing/templates.html>

Your report must include

1. Introduction
2. Mathematical formulation
3. Numerical algorithm
4. Verification and testing
5. Profiling results

6. Parallel implementation
7. Performance analysis
8. Discussion and conclusions

## 16 Submission

Submit the following:

- Source code (Fortran or C++) (over github)
- Build instructions (as a makefile)
- Plots and figures (in publication quality)
- IEEE-format white paper (PDF)

## 17 Evaluation Criteria

Your assignment will be evaluated based on

- correctness of the numerical implementation
- verification tests
- quality of parallelisation
- performance analysis
- clarity of the white paper

## 18 Bonus Extension: Neighbour Search (optional for additional 1 mark)

**Optional bonus task:** Implement a basic neighbour-search strategy to reduce the cost of particle–particle contact detection.

In the baseline implementation, checking all particle pairs requires

$$O(N^2)$$

operations per timestep. For larger systems, this may become the dominant computational cost.

One possible improvement is to divide the computational domain into a uniform Cartesian grid of cells and assign each particle to a cell. Contact checks for a particle then need only be performed with particles in the same cell and adjacent cells.

You may implement a simple cell-linked list or equivalent neighbour-search strategy.

Compare the neighbour-search implementation against the brute-force all-pairs approach in terms of



- correctness of detected contacts
- total runtime
- runtime of the contact-search stage
- number of candidate pairs examined
- scaling with particle number

Discuss clearly whether the neighbour-search method improves performance for all problem sizes, and explain any overheads observed for small systems.

## 19 Bonus Scientific Investigation (Optional for additional 1 mark)

Students may perform additional numerical experiments to explore the physical and numerical behaviour of the DEM solver. These investigations should resemble small research-style studies and may earn additional credit.

Possible studies include:

### 19.1 Effect of Damping

Drop a single particle onto the floor and vary the damping coefficient  $\gamma_n$ . Measure the rebound height after each bounce and analyse the rate of energy dissipation.

Suggested plots:

- particle height vs time
- rebound height vs bounce number
- kinetic energy vs time

Finally present results for emergent behaviour of the system of particles in box as a function of the damping coefficient.

### 19.2 Timestep Sensitivity

Investigate how the timestep size  $\Delta t$  affects the accuracy and stability of the simulation. Compare results for several values of  $\Delta t$ .

Suggested plots:

- numerical vs analytical trajectory
- error vs timestep
- energy evolution

### 19.3 Particle Cloud Settling

Initialise a collection of particles in the box with zero initial velocity and allow them to settle under gravity.

Analyse the evolution of the system.

Suggested quantities:

- centre of mass height
- total kinetic energy
- particle configurations at selected times

## 20 Bonus Research Extension (Optional for maximum 5 additional marks)

Highly motivated students may extend this project into a small research-style study and submit an **additional** full paper describing their investigation.

This component is optional and intended for students who wish to explore scientific or computational aspects of the DEM solver in greater depth. Students choosing this option may receive additional marks depending on the quality and depth of the investigation. Also, the marks will only be awarded if the paper is publication quality. Discuss with the instructor if in doubt.

### 20.1 Goal

The goal of the extension is to conduct a systematic computational study using the developed DEM solver. The study should resemble a small research project that investigates either

- numerical behaviour of the algorithm,
- physical behaviour of granular systems, or
- computational performance of the solver.

Students may work individually or in small groups (maximum two students).

### 20.2 Paper Requirements

Students undertaking this extension must submit a single **15–20 page paper** prepared using the **IEEE conference format**; this is in addition to the white paper.

The paper must contain the following sections:

1. Introduction (including a brief literature review)
2. Mathematical model
3. Numerical algorithm
4. Code implementation and performance analysis (serial and parallel)

5. Testing and verification
6. Experimental setup
7. Results and analysis
8. Discussion
9. Conclusions

The paper should clearly state the objective of the study and present results in a form similar to a computational research article. All details included above in the main assignment should also be included in the paper.

## 20.3 Suggested problems

You may choose out of these problems or one of your own (discuss with the instructor first).

### 20.3.1 Problem 1: Dense Bed Impact

A dense granular bed is first formed at the bottom of the domain. A compact block of particles is then released above the bed and allowed to impact it.

#### Initial Setup

1. Generate a dense bed of particles in the region

$$0 < x < L_x, \quad 0 < y < L_y, \quad 0 < z < H_b$$

2. Allow the bed to settle under gravity until motion becomes small.
3. Place an additional block of particles above the bed

$$x_a < x < x_b, \quad y_a < y < y_b, \quad z_a < z < z_b$$

and allow it to fall under gravity.

#### Suggested Studies

Students may vary

- impact height
- size of the falling particle block
- damping coefficient

## Diagnostics

- centre-of-mass height of the impacting particles
- total kinetic energy of the system
- penetration depth of the impacting block
- number of active contacts in the system

## Required Figures

- configuration snapshots during impact
- centre-of-mass height of impactor vs time
- kinetic energy vs time
- penetration depth vs parameter variations

### 20.3.2 Problem 2: Granular Motion on an Undulating Slope

In this experiment the flat floor is replaced by a terrain surface that contains slope and undulations. This setup mimics particle motion over irregular terrain.

#### Terrain Surface

The terrain may be defined analytically as

$$z_b(x, y) = z_0 + s_x x + A \sin\left(\frac{2\pi x}{\lambda_x}\right) \sin\left(\frac{2\pi y}{\lambda_y}\right)$$

where

- $s_x$  is the overall slope
- $A$  is the amplitude of terrain undulations
- $\lambda_x, \lambda_y$  are wavelengths

Particles are released near the top of the slope and allowed to move downslope under gravity.

#### Suggested Studies

Students may vary

- slope magnitude  $s_x$
- terrain amplitude  $A$
- terrain wavelength

## Diagnostics

- centre-of-mass position along the slope
- particle runout distance
- total kinetic energy vs time
- particle spreading across the terrain

## Required Figures

- terrain surface visualisation
- particle snapshots moving downslope
- centre-of-mass position vs time
- runout distance vs terrain parameters

### 20.3.3 Problem 3: Terrain Obstacle Interaction

In this case a localized obstacle (representing a hill or mound) is placed on an inclined surface. Particles released upslope interact with the obstacle.

## Terrain Model

The surface may be defined as

$$z_b(x, y) = z_0 + s_x x + A \exp \left( -\frac{(x - x_c)^2}{\sigma_x^2} - \frac{(y - y_c)^2}{\sigma_y^2} \right)$$

where the exponential term represents a smooth mound.

## Suggested Studies

Students may vary

- obstacle height  $A$
- obstacle width  $\sigma_x, \sigma_y$
- slope angle

## Diagnostics

- particle runout distance
- lateral spreading of particles
- time required for particles to reach rest
- final deposit shape

### Required Figures

- initial particle configuration
- particle trajectories interacting with the obstacle
- runout distance vs obstacle height
- final particle distribution

## 20.4 Paper Evaluation

The additional paper will be evaluated based on

- clarity of the research objective
- quality of numerical experiments
- depth of analysis and interpretation
- quality of figures and visualisations
- discussion of computational performance

Students should clearly explain both the **physical behaviour of the granular system** and the **computational characteristics of their implementation**.